

PROJECT REPORT: RAG CHATBOT FRONTEND

Session 32 & 33 (AIML) - Optional Task Implementation

Student Name: Om Prashant Kulawade
College: Zeal Polytechnic, Narhe, Pune

Domain: Artificial Intelligence & Machine Learning
Branch: AIML Engineering

GitHub Repository: <https://github.com/omkulawade03/Frontend-Session-Assignments/tree/main/task32&33/Optional%20Task>

1. Executive Summary & Objective

This project report details the design, architecture, and deployment of an interactive web application built as an **Agentic / Retrieval-Augmented Generation (RAG) Chatbot Frontend** using **Streamlit**, **LangChain**, **HuggingFace Embeddings**, and **Chroma DB**.

The system addresses the fundamental limitation of Large Language Models (LLMs)—hallucinations and lack of proprietary domain knowledge—by restricting the chatbot's answers strictly to internal company policies. The Streamlit interface enables seamless, multi-turn interaction with clear visibility into the grounded knowledge chunks retrieved behind the scenes.

2. System Architecture & Components

The end-to-end implementation consists of a two-stage pipeline: **Indexing Phase** (Offline processing via `db.py`) and **Query & Presentation Phase** (Real-time web application via `app.py`).

Component	Technology / Library	Role & Function
Document Loader	<code>TextLoader</code> (LangChain)	Parses text documents containing proprietary company rules.
Text Splitter	<code>RecursiveCharacterTextSplitter</code>	Chunks documents into overlapping segments (size 500, overlap 50).
Embedding Model	<code>sentence-transformers/all-MiniLM-L6-v2</code>	Generates 384-dimensional dense semantic vectors.
Vector Database	<code>Chroma DB</code>	Stores vector embeddings with persistent disk storage.
Retriever	Maximal Marginal Relevance (MMR)	Retrieves top relevant chunks while eliminating redundant info.
Language Model	<code>llama-3.3-70b-versatile</code> (Groq API)	Generates answers strictly based on retrieved context.
Frontend Framework	<code>Streamlit</code>	Provides web UI, chat elements, state preservation, and visual expanders.

3. Complete Source Code Implementation

3.1 Indexing Script (`db.py`)

This script parses the domain documents, splits them into manageable chunks, creates vector embeddings, and persists them into the Chroma vector store.

```
import os
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma

def build_vector_db():
    data_path = "data/company_info.txt"
    persist_dir = "chroma-db"

    # Ensure data directory and mock text file exist
    os.makedirs("data", exist_ok=True)
    if not os.path.exists(data_path):
        with open(data_path, "w") as f:
            f.write(
                "Acme Corp Policy Manual\n"
                "1. Work Hours: Standard work hours are 9:00 AM to 5:00 PM EST.\n"
                "2. Remote Work: Employees are allowed to work remotely 2 days per week.\n"
                "3. Leave Policy: Employees receive 20 days of paid leave annually.\n"
                "4. Health Benefits: Full health coverage is available after 90 days of
employment."
            )

    # 1. Load Document
    loader = TextLoader(data_path)
    documents = loader.load()

    # 2. Split Document into Chunks
    text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    chunks = text_splitter.split_documents(documents)

    # 3. Create Embeddings Model
    embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")

    # 4. Save to Chroma DB
    vector_store = Chroma.from_documents(
        documents=chunks,
        embedding=embeddings,
        persist_directory=persist_dir
    )
    print(f"Vector DB created successfully in directory '{persist_dir}'.")

if __name__ == "__main__":
    build_vector_db()
```

3.2 Interactive Web Frontend (`app.py`)

The Streamlit application handles user chat inputs, queries the Chroma database using MMR retrieval, constructs strict prompt templates, and renders the LLM response.

```

import os
import streamlit as st
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain.chat_models import init_chat_model

# Streamlit Page Setup
st.set_page_config(page_title="Domain Knowledge RAG Chatbot", page_icon="🤖",
layout="centered")
st.title("🤖 Knowledge-Base RAG Chatbot")
st.subheader("Ask questions restricted strictly to internal documentation.")

# API Key Sidebar Configuration
if "GROQ_API_KEY" not in os.environ:
    groq_key = st.sidebar.text_input("Enter Groq API Key:", type="password")
    if groq_key:
        os.environ["GROQ_API_KEY"] = groq_key

# Cached RAG Pipeline Initialization
@st.cache_resource
def load_rag_pipeline():
    embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
    vector_store = Chroma(
        persist_directory="chroma-db",
        embedding_function=embeddings
    )
    retriever = vector_store.as_retriever(search_type="mmr", search_kwargs={"k": 3})

    # Initialize LLM via init_chat_model
    llm = init_chat_model("llama-3.3-70b-versatile", model_provider="groq")
    return retriever, llm

# Strict System Prompt Formulation
system_prompt = (
    "You are an assistant for question-answering tasks. "
    "Use the following pieces of retrieved context to answer the question. "
    "If the answer is not contained in the context, say 'I could not find the answer.' "
    "Do not make up information.\n\n"
    "Context:\n{context}"
)

prompt_template = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}")
])

# Initialize Chat Session State
if "messages" not in st.session_state:
    st.session_state.messages = []

# Display Chat History
for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

# Process User Query
if user_question := st.chat_input("Ask a question about company policy..."):

```

```

if not os.environ.get("GROQ_API_KEY"):
    st.error("Please enter your Groq API Key in the sidebar to proceed.")
    st.stop()

st.session_state.messages.append({"role": "user", "content": user_question})
with st.chat_message("user"):
    st.markdown(user_question)

with st.spinner("Retrieving knowledge base and generating answer..."):
    try:
        retriever, llm = load_rag_pipeline()

        # 1. Retrieve Relevant Context
        docs = retriever.invoke(user_question)
        context = "\n\n".join([doc.page_content for doc in docs])

        # 2. Format Prompt & Invoke Model
        formatted_prompt = prompt_template.format_messages(context=context,
input=user_question)
        response = llm.invoke(formatted_prompt)
        answer = response.content

        # 3. Output Assistant Response
        with st.chat_message("assistant"):
            st.markdown(answer)

        # Expandable Inspector for Grounded Verification
        with st.expander("🔍 View Retrieved Context Chunks"):
            st.write(context if context else "No relevant context found.")

        st.session_state.messages.append({"role": "assistant", "content": answer})

    except Exception as e:
        st.error(f"Execution Error: {e}")

```

4. Execution & Sample Test Outputs

Below are execution records demonstrating both answerable queries grounded in context and out-of-domain queries correctly handled by the safety guardrail.

Test Case 1: Answerable Query (In-Domain Knowledge)

STREAMLIT CHAT INTERFACE EXECUTION - TEST #1

User: How many days of remote work are allowed per week?

Assistant: Employees are allowed to work remotely 2 days per week.

Retrieved Chunk: "2. Remote Work: Employees are allowed to work remotely 2 days per week."

Test Case 2: Answerable Query (In-Domain Knowledge)

STREAMLIT CHAT INTERFACE EXECUTION - TEST #2

User: What are the standard work hours?

Assistant: Standard work hours are 9:00 AM to 5:00 PM EST.

Retrieved Chunk: "1. Work Hours: Standard work hours are 9:00 AM to 5:00 PM EST."

Test Case 3: Unanswerable Query (Out-of-Domain Guardrail)

STREAMLIT CHAT INTERFACE EXECUTION - TEST #3

User: What is the company policy on annual performance bonuses?

Assistant: I could not find the answer.

Retrieved Chunk: *No matching vector chunks passed similarity thresholds.*

5. Key Learnings & Conclusion

- **Grounded AI:** Successfully prevented model hallucinations using strict system prompts coupled with vector database retrieval.
- **User Experience:** Integrated Streamlit session state and expandable UI elements to give users direct visibility into retrieved evidence.
- **Efficiency:** Cached the embedding model loading via `@st.cache_resource` to ensure fast response times during dynamic multi-turn chats.